

Programación de sistemas y procesos

2º DAM - UD 5 - Caso practico 1

José Cruces Aparicio

· TÍTULO

Servicio SSL.

· SITUACIÓN

En la empresa donde trabajamos, tenemos un software que envía una serie de datos a unos dispositivos hardware. La arquitectura de trabajo es cliente-servidor:

```
import java.io.PrintStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Calendar;
import java.util.GregorianCalendar;
import java.util.Scanner;

public class TimeServer {

    private static final String[] nombreDias =
{"Domingo", "Lunes", "Martes",
"Miércoles", "Jueves", "Viernes",
                                                                    "Sábado"};

    public static void main(String[] argv) throws
Exception {

        try {
            ServerSocket srvSocket = new
ServerSocket(10000);

            while (true) {
                Socket cliSocket = srvSocket.accept();

                Scanner in = new Scanner
(cliSocket.getInputStream());
                int[] data = new int[3];
```

```

        boolean ok = true;
        for (int i = 0; i < data.length; i++) {
            if (in.hasNextInt()) {
                data[i] = in.nextInt();
            } else {
                ok = false;
            }
        }

        PrintStream out = new
        PrintStream(cliSocket.getOutputStream());
        if (ok) {
            data[1] -= 1;
            GregorianCalendar cal = new
            GregorianCalendar(data[2], data[1], data[0]);
            int dia = cal.get(Calendar.DAY_OF_WEEK) - 1;
            out.println("Este dia es " + nombreDias[dia]
            + ".");
        } else {
            out.println("Formato de datos incorrecto.");
        }
        cliSocket.close();
    }
    } catch (Exception ex) {
        System.out.println("Error en la comunicacion: "
        + ex);
    }
}
}

```

• INSTRUCCIONES

1. Averiguad qué hace el servicio y generad un cliente no seguro que se comunique correctamente. Su clase se puede denominar TimeClient.
2. Generad dos nuevas clases Java denominadas SecureTimeServer y Secure TimeClient que adapten el cliente y el servicio al protocolo SSL con autenticación simple. Todos los ficheros asociados a claves y certificados se guardan en la carpeta SSL, tanto al cliente como al servicio.

1. Análisis del funcionamiento del servicio

El sistema está basado en una arquitectura **cliente-servidor TCP**, donde el servidor escucha conexiones entrantes y el cliente envía datos para ser procesados.

Funcionamiento del servidor original

El servidor realiza las siguientes operaciones:

1. Abre un `ServerSocket` en el puerto **10000**
2. Espera conexiones de clientes mediante `accept()`
3. Recibe **tres valores enteros**:
 - Día
 - Mes
 - Año
4. Valida que los datos sean numéricos
5. Ajusta el mes (-1) porque `GregorianCalendar` empieza en 0
6. Calcula el día de la semana
7. Devuelve el nombre del día en español

Lógica interna

Usa `GregorianCalendar` para el cálculo de fechas

Usa `Calendar.DAY_OF_WEEK` para obtener el día

Devuelve una cadena con el resultado

Resumen del protocolo

El cliente debe enviar:

día mes año

Ejemplo:

25 12 2024

Respuesta esperada:

Este día es Miércoles.

2. Cliente no seguro (TimeClient)

Este cliente utiliza sockets TCP sin cifrado para comunicarse con el servidor.

Implementación correcta

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class TimeClient {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 10000;

        try (Socket socket = new Socket(host, port);
            PrintStream out = new
PrintStream(socket.getOutputStream());
            Scanner in = new
Scanner(socket.getInputStream())) {

            // Envío correcto de datos como enteros
independientes
            out.println(25);
            out.println(12);
            out.println(2024);

            // Lectura de respuesta
            if (in.hasNextLine()) {
```

```
                System.out.println("Respuesta del servidor: "
+ in.nextLine());
            }

            } catch (IOException e) {
                System.err.println("Error en el cliente: " +
e.getMessage());
            }
        }
    }
}
```

Observación

Este cliente funciona correctamente, pero no ofrece seguridad:

Datos en texto plano

Vulnerable a interceptación (sniffing)

3. Adaptación a SSL (Secure Socket Layer)

Problema del sistema original

El sistema TCP transmite datos sin cifrar, lo que permite:

Interceptación de datos

Manipulación del tráfico

Falta de autenticación del servidor

Solución

Se implementa **SSL con autenticación simple**, donde:

Solo el servidor se autentica (certificado del servidor)

El cliente confía en el servidor mediante truststore

Generación de certificados

En la carpeta SSL:

```
keytool -genkeypair -alias serverKey -keyalg RSA -keystore  
servidor.jks -storepass password
```

Esto crea:

`servidor.jks` → almacén de claves del servidor

4. Servidor seguro (SecureTimeServer)

```
import javax.net.ssl.*;  
import java.io.*;  
import java.util.*;  
  
public class SecureTimeServer {  
  
    private static final String[] nombreDias = {  
        "Domingo", "Lunes", "Martes",  
        "Miércoles", "Jueves", "Viernes", "Sábado"  
    };  
  
    public static void main(String[] args) throws Exception {  
  
        // Configuración SSL  
        System.setProperty("javax.net.ssl.keyStore", "SSL/  
servidor.jks");  
        System.setProperty("javax.net.ssl.keyStorePassword",  
"password");  
  
        SSLServerSocketFactory ssf =  
            (SSLServerSocketFactory)  
SSLServerSocketFactory.getDefault();  
  
        SSLServerSocket serverSocket =  
            (SSLServerSocket)  
ssf.createServerSocket(10000);
```

```

        System.out.println("Servidor SSL iniciado...");

        while (true) {
            try (SSLSocket clientSocket = (SSLSocket)
serverSocket.accept());
                Scanner in = new
Scanner(clientSocket.getInputStream());
                PrintStream out = new
PrintStream(clientSocket.getOutputStream())) {

                    int[] data = new int[3];
                    boolean ok = true;

                    for (int i = 0; i < 3; i++) {
                        if (in.hasNextInt()) {
                            data[i] = in.nextInt();
                        } else {
                            ok = false;
                        }
                    }

                    if (ok) {
                        data[1] -= 1;

                        GregorianCalendar cal =
                            new GregorianCalendar(data[2],
data[1], data[0]);

                        int dia = cal.get(Calendar.DAY_OF_WEEK) -
1;

                        out.println("Este día es " +
nombreDias[dia]);
                    } else {
                        out.println("Formato de datos
incorrecto.");
                    }
                }
            }
        }
    }
}

```

5. Cliente seguro (SecureTimeClient)

```
import javax.net.ssl.*;
import java.io.*;
import java.util.Scanner;

public class SecureTimeClient {

    public static void main(String[] args) throws Exception {

        // Truststore del servidor
        System.setProperty("javax.net.ssl.trustStore", "SSL/
servidor.jks");

        System.setProperty("javax.net.ssl.trustStorePassword",
"password");

        SSLSocketFactory sf =
            (SSLSocketFactory)
        SSLSocketFactory.getDefault();

        try (SSLSocket socket =
            (SSLSocket) sf.createSocket("localhost",
10000);
            PrintStream out = new
        PrintStream(socket.getOutputStream());
            Scanner in = new
        Scanner(socket.getInputStream())) {

            // Envío de datos
            out.println(20);
            out.println(4);
            out.println(2026);

            // Respuesta del servidor
            System.out.println("Servidor seguro dice: " +
in.nextLine());
        }
    }
}
```

6. Explicación del funcionamiento SSL

Qué aporta SSL

- Cifrado de datos (confidencialidad)
- Integridad de la información
- Autenticación del servidor

Autenticación simple

En este caso:

Solo el servidor se autentica mediante certificado

El cliente confía en el servidor usando `trustStore`

Flujo de comunicación

Cliente → conexión SSL → Servidor

↓

canal cifrado

↓

Intercambio seguro de datos

7. Conclusión

La migración de un sistema TCP a SSL permite mejorar significativamente la seguridad del sistema sin modificar la lógica de negocio.

La arquitectura resultante garantiza:

Comunicación cifrada

Mayor seguridad frente a interceptación

Integridad de los datos enviados

8. Bibliografía

Apuntes de la Unidad 5 – Programación de Sistemas y Procesos

Documentación oficial de Java:

SSLSocket / SSLServerSocket

java.security.KeyStore

Oracle Java Tutorials – Networking & SSL